

Java 8 Stream API Interview Cheat Sheet

1. Common Stream Methods

`filter()` — Keep matching elements.
`map()` — Transform each element.
`flatMap()` — Flatten nested collections.
`distinct()` — Remove duplicates.
`sorted()` — Sort ascending or with a comparator.
`limit(n)` — Keep first N elements.
`skip(n)` — Skip first N elements.
`collect()` — Convert to List, Set, Map, etc.
`count()` — Count elements.
`findFirst()` — Return first matching element.
`findAny()` — Return any matching element.
`max()` — Largest element.
`min()` — Smallest element.
`anyMatch()` — True if any element matches.
`allMatch()` — True if all elements match.
`noneMatch()` — True if no element matches.
`mapToInt()` — Convert to `IntStream`.
`sum()` — Sum numeric values.

2. Common Collectors

`Collectors.toList()`
`Collectors.toSet()`
`Collectors.toCollection(TreeSet::new)`
`Collectors.joining(", ")`
`Collectors.groupingBy()`
`Collectors.partitioningBy()`
`Collectors.counting()`

3. Stream Flow

Collection -> `stream()` -> Intermediate Operations (`filter/map/sorted/distinct`) -> Terminal Operation (`collect/count/findFirst`)

Important Coding Examples

1. Remove Duplicates

```
List result = nums.stream()
    .distinct()
    .toList();
```

2. Find Even Numbers

```
List result = nums.stream()
    .filter(n -> n % 2 == 0)
    .toList();
```

3. Square Every Number

```
List result = nums.stream()
    .map(n -> n * n)
    .toList();
```

4. Find Maximum

```
int max = nums.stream()
    .max(Integer::compare)
    .get();
```

5. Sum of Numbers

```
int sum = nums.stream()
    .mapToInt(Integer::intValue)
    .sum();
```

6. Sort Descending

```
List result = nums.stream()
    .sorted(Collections.reverseOrder())
    .toList();
```

7. Top 3 Largest Numbers

```
List result = nums.stream()
    .sorted(Collections.reverseOrder())
    .limit(3)
    .toList();
```

8. Join Strings

```
String result = names.stream()
    .collect(Collectors.joining(", "));
```

9. Flatten Comma-Separated Strings

```
List result = list.stream()
    .flatMap(s -> Arrays.stream(s.split(",")))
    .map(Integer::parseInt)
    .toList();
```

10. Remove Duplicates & Sort Descending

```
List result = list.stream()
    .flatMap(s -> Arrays.stream(s.split(",")))
    .map(Integer::parseInt)
    .distinct()
    .sorted(Collections.reverseOrder())
    .toList();
```

11. Group Employees by Department

```
Map> result = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));
```

12. Count Employees by Department

```
Map result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.counting()));
```

13. Highest Salary Employee

```
Employee result = employees.stream()
    .max(Comparator.comparing(Employee::getSalary))
    .get();
```

14. Second Highest Number

```
int secondHighest = nums.stream()
    .distinct()
    .sorted(Collections.reverseOrder())
    .skip(1)
    .findFirst()
    .get();
```

15. Partition Even & Odd Numbers

```
Map> result = nums.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

16. anyMatch Example

```
boolean exists = nums.stream()
    .anyMatch(n -> n > 100);
```